Applied Optimization
Assignment Week Three

First Name	
Last Name	
ID#	
Email Address	

Table of Contents

How to submit your Assignment 1
P1 [15 points] 2
P2 [15 points] 2
P3 [15 points] 4
P4 [15 points] 5
P5 [20 points] 7
P6 [20 points] 9

How to submit your Assignment

After filling all the parts in this file, please follow the following steps.

1) Add your name and ID to the first page.
2) Save the file in the original format (Docx or Doc)

   (please do not convert to other file formats e.g. PDF, ZIP, RAR, …).

3) Rename the file as

   *YOUR **First** Name* - *YOUR **Last** Name* - *ID* – ANA670 – ***HW3**.docx*

   Example: John - Smith - ID - ANA670 – HW3.docx

4) Upload the file and submit it (only using BrightSpace)

P1 [15 points]

Consider the case of a 2D random walk where each step is of fixed unit length. Derive the distance traveled after N steps and its variance using the complex numbers on a plane.

Solution

ANA670 - Week 3 HW - Q1

Assumptions:

> $\cos^2\theta + \sin^2\theta = 1$
> $\sqrt{-1} = i$
> $i^2 = -1$
> N = number of steps

Total position as complex number: $Z = x + iy = r(\cos\theta + i\sin\theta)$
Pythagorean Theorem (PT): $R^2 = x^2 + y^2$
PT with complex numbers using conjugate, ($\overline{Z} = x - iy$): $R^2 = Z \times \overline{Z} = (x + iy)(x - iy) = x^2 + y^2$

Distance using complex numbers:
$R^2 = (z_1 + z_2 + z_3 + \cdots + z_N) \times (\overline{z_1} + \overline{z_2} + \overline{z_3} + \cdots + \overline{z_N})$

> $z_1\overline{z_1} = (\cos\theta + i\sin\theta)(\cos\theta - i\sin\theta)$
> $z_1\overline{z_1} = \cos^2\theta + i\cos\theta\sin\theta - i\cos\theta\sin\theta - i^2\sin^2\theta$
> $z_1\overline{z_1} = \cos^2\theta + \sin^2\theta = 1$ $z_1\overline{z_2} = (\cos\theta_1 + i\sin\theta_1)(\cos\theta_2 - i\sin\theta_2)$
> $z_1\overline{z_2} = \cos\theta_1\cos\theta_2 - i\cos\theta_1\sin\theta_2 + i\sin\theta_1\cos\theta_2 - i^2\sin\theta_1\sin\theta_2$
> $z_1\overline{z_2} = (\cos\theta_1\cos\theta_2 + \sin\theta_1\sin\theta_2) + i(\sin\theta_1\cos\theta_2 - \cos\theta_1\sin\theta_2)$
> $z_1\overline{z_2} = \cos(\theta_1 - \theta_2) + i\sin(\theta_1 - \theta_2)$
> $E[z_1\overline{z_2}] = 0$ (average over many walks)

$E[R^2] = 1 + 1 + 1 + \cdots + 1 = N$
$\mathbf{R = \sqrt{N}}$

$\mathrm{Var}(R^2) = E[R^4] - (E[R^2])^2 = (2N^2 - N) - N^2 = N(N-1)$

P2 [15 points]

Write a simple program to carry out some Levy flights and demonstrate how the exponent ß affects the distance traveled.

$$\sigma^2(t) \sim \begin{cases} t^2 & \text{if } 0 < \beta < 1 \\ t^{3-\beta} & \text{if } 1 < \beta < 2 \\ t & \text{if } \beta \geq 2 \end{cases},$$

Your Code

```
# ANA670 - Week 3 HW - Q2.1
steps, s0 = 30, 1.0
beta_1 = 0.5
beta_2 = 1.5
beta_3 = 2.5

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
regimes = [
```

```
    (beta_1, 'r', r'$\sigma^2(t) \sim t^2$', r'$\sigma^2(t) \sim t^2$ ($0 < \beta_1 = ' + str(beta_1) + r' < 1$)'),
    (beta_2, 'g', r'$\sigma^2(t) \sim t^{3-\beta}$', r'$\sigma^2(t) \sim t^{3-' + str(beta_2) + r'}$ ($1 < \beta_2 = ' + str(beta_2) + r' < 2$)'),
    (beta_3, 'b', r'$\sigma^2(t) \sim t$', r'$\sigma^2(t) \sim t$ ($\beta_3 = ' + str(beta_3) + r' \geq 2$)')
]

for ax, (beta, color, eq, title) in zip(axes, regimes):
    np.random.seed(42)
    u = np.random.uniform(0, 1, steps)
    s = np.random.normal(s0, 0.3, steps) if beta >= 2 else s0 / (u ** (1 / beta))
    angles = np.random.uniform(0, 2 * np.pi, steps)
    dx, dy = s * np.cos(angles), s * np.sin(angles)
    x = np.cumsum(np.insert(dx, 0, 0))
    y = np.cumsum(np.insert(dy, 0, 0))

    ax.plot(x, y, color=color, alpha=0.6, linewidth=2, label=eq)
    ax.set_title(title); ax.axis('equal'); ax.grid(True, linestyle=':'); ax.legend(loc='upper left')

plt.suptitle('Levy Flight Scaling Comparison', fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

# ANA670 - Week 3 HW - Q2.2
import numpy as np
import matplotlib.pyplot as plt

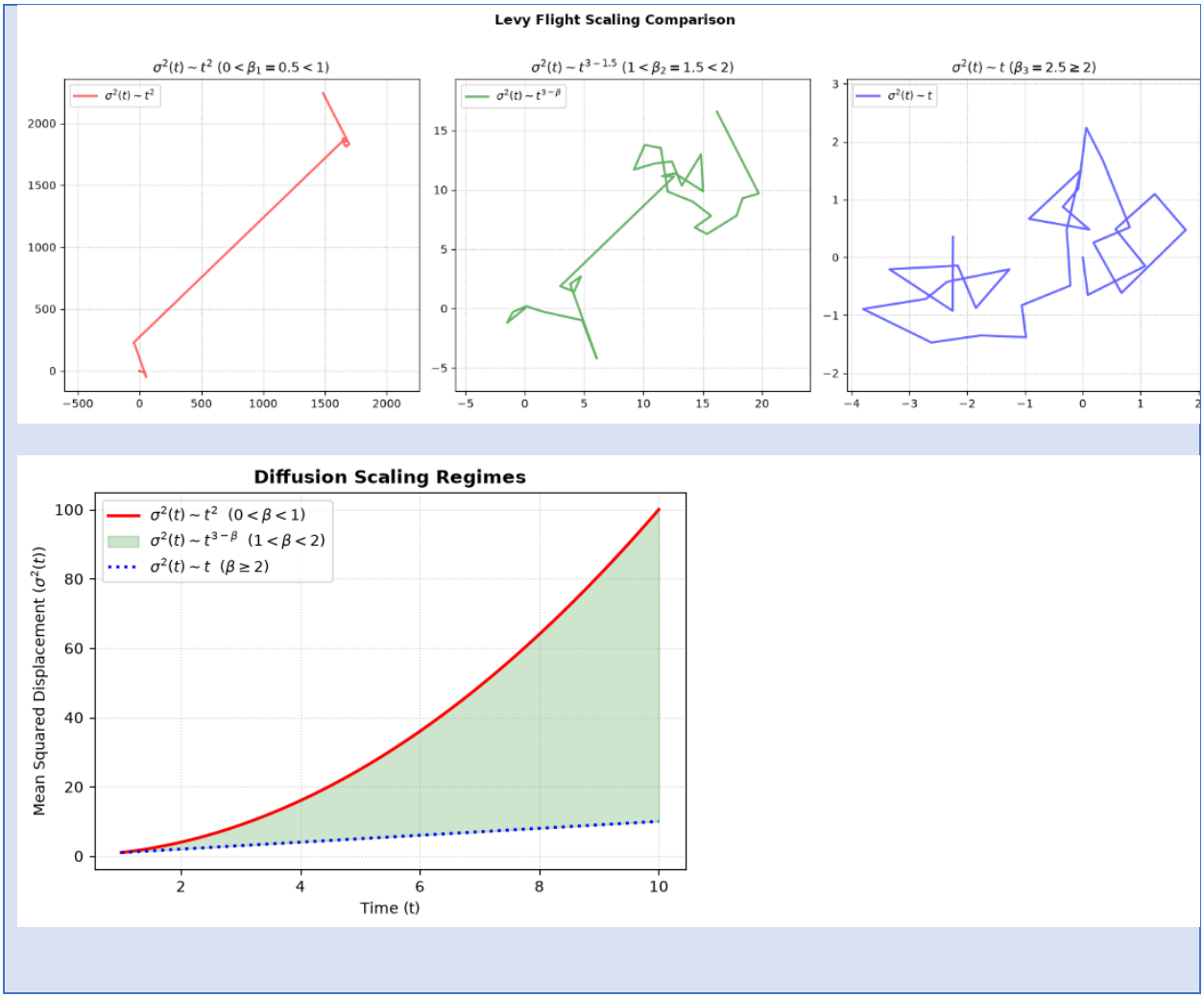
t = np.linspace(1, 10, 100)
y_upper, y_lower = t**2, t

plt.figure(figsize=(7, 4.5))
plt.plot(t, y_upper, 'r-', linewidth=2, label=r'$\sigma^2(t) \sim t^2$  ($0 < \beta < 1$)')
plt.fill_between(t, y_lower, y_upper, color='g', alpha=0.2, label=r'$\sigma^2(t) \sim t^{3-\beta}$  ($1 < \beta < 2$)')
plt.plot(t, y_lower, 'b:', linewidth=2, label=r'$\sigma^2(t) \sim t$  ($\beta \geq 2$)')

plt.title('Diffusion Scaling Regimes', fontweight='bold')
plt.xlabel('Time (t)')
plt.ylabel(r'Mean Squared Displacement ($\sigma^2(t)$)')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='upper left')
plt.show()
```

Run the code and insert the result in the following box.

Plots



P3 [15 points]

Write a simple program to generate of a random walk.

Your Code

```
# ANA670 - Week 3 HW - Q3
np.random.seed(42)
num_walks = 1000
num_steps = 10000
choices = np.random.randint(0, 4, size=(num_steps, num_walks))
moves_map = np.array([[0, 1], [0, -1], [1, 0], [-1, 0]])
steps = moves_map[choices]

walks = np.zeros((num_steps + 1, num_walks, 2))
walks[1:] = np.cumsum(steps, axis=0)

final_positions = walks[-1]
distances = np.sqrt(final_positions[:, 0]**2 + final_positions[:, 1]**2)

print(f"Average Final Distance from Origin : {np.mean(distances):.2f} units")
```

```
print(f"Max Distance Reached by any Path   : {np.max(distances):.2f} units")
print(f"Theoretical RMS Distance (√N)      : {np.sqrt(num_steps):.2f} units")
print(f"Theoretical Mean Distance          : {np.sqrt(num_steps) * (np.sqrt(np.pi) / 2):.2f} units")

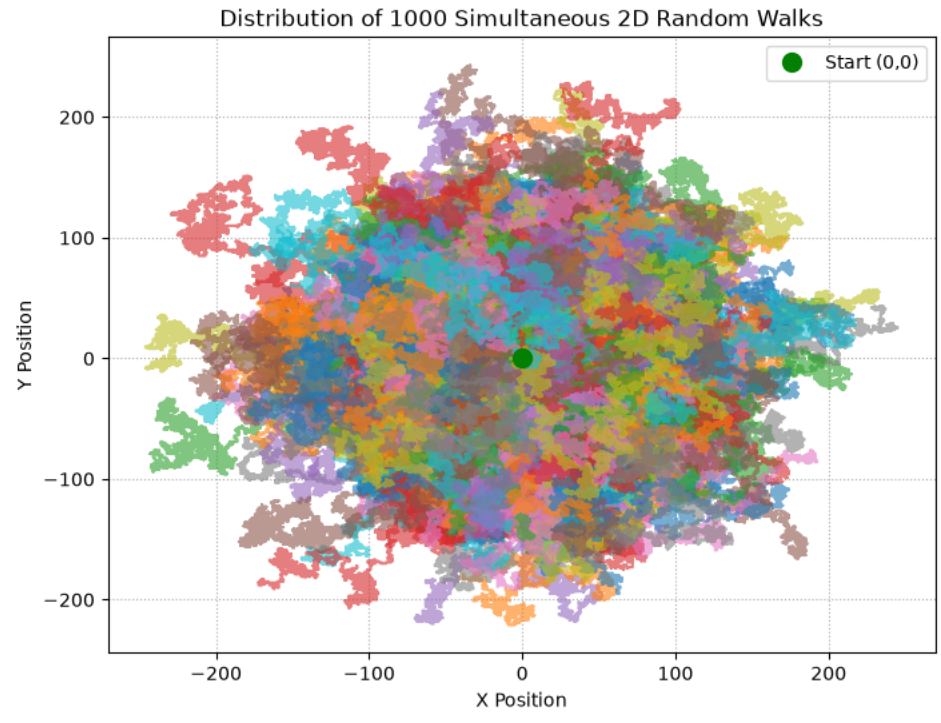

plt.figure(figsize=(8, 6))
for i in range(num_walks):
    plt.plot(walks[:, i, 0], walks[:, i, 1], alpha=0.6)

plt.plot(0, 0, 'go', markersize=10, label='Start (0,0)')
plt.title(f"Distribution of {num_walks} Simultaneous 2D Random Walks")
plt.xlabel("X Position"); plt.ylabel("Y Position"); plt.grid(True, linestyle=":")
plt.legend()
plt.show()
```

Run the code and insert the result in the following box.

Plots

```
Average Final Distance from Origin : 88.34 units
Max Distance Reached by any Path   : 253.40 units
Theoretical RMS Distance (√N)      : 100.00 units
Theoretical Mean Distance          : 88.62 units
```



P4 [15 points]

The method of least-squares works well for most data-fitting problem; however, for polynomials

$$y = f(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \ldots + \alpha_p x^p,$$

it tends to result in high-order polynomials to best-fit the given data set (x_i, y_i) where $i = 1, 2,..., n$, even though the higher-order polynomials are not necessarily the best model. A way to improve this is to use some penalty in coefficients so as to balance the goodness of the fit and the choice of reasonably simple model. Design an optimization problem to achieve this by limiting the sum of the coefficients α_k where $k = 0, 1 , ..., p$.

Solution

```
# ANA670 - Week 3 HW - Q4
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error

np.random.seed(42)
noise = 0.3
n_points = 15
rng = 2
X = np.linspace(-rng, rng, n_points).reshape(-1, 1)
y = (0.5 * X**3 - X + np.random.normal(0, noise, size=X.shape)).flatten()
degree = 6
lam = 0.5

fits = {
    'Unoptimized (OLS)': LinearRegression(fit_intercept=False),
    'L1 (Lasso)': Lasso(alpha=lam, fit_intercept=False, max_iter=50000, tol=1e-2),
    'L2 (Ridge)': Ridge(alpha=lam, fit_intercept=False, solver='svd')
}

def fmt_eq(c):
    t = []
    if abs(c[0]) > 1e-4: t.append(f"{c[0]:.4f}")
    for i in range(1, len(c)):
        if abs(c[i]) > 1e-4:
            s = " + " if c[i] > 0 else " - "
            v = abs(c[i])
            if not t: t.append(f"{'-' if s == ' - ' else ''}{v:.4f}*x^{i}")
            else: t.append(f"{s.strip()} {v:.4f}*x^{i}")
    return "y = " + " ".join(t) if t else "y = 0.0000"

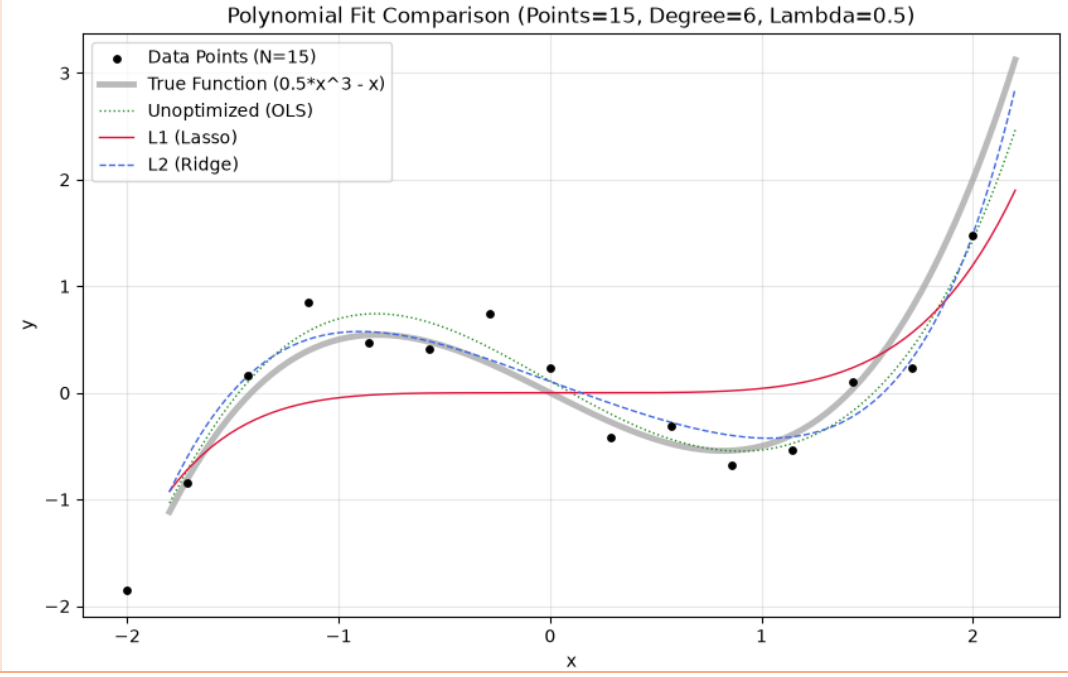
X_pl = np.linspace(-rng+0.2, rng+0.2, 200).reshape(-1, 1)
y_true = 0.5 * X_pl**3 - X_pl
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='black', s=15, zorder=5, label=f'Data Points (N={n_points})')
plt.plot(X_pl, y_true, color='darkgray', linestyle='-', linewidth=3.5, alpha=0.8, label='True Function
(0.5*x^3 - x)')

colors, styles = ['forestgreen', 'crimson', 'royalblue'], [':', '-', '--']
for (name, model), color, style in zip(fits.items(), colors, styles):
    pipe = make_pipeline(PolynomialFeatures(degree, include_bias=True), model).fit(X, y)
```

```
    c = pipe.named_steps[model.__class__.__name__.lower()].coef_
    mse = mean_squared_error(y, pipe.predict(X))
    print(f"{name}: {fmt_eq(c)} | MSE: {mse:.4f}")
    plt.plot(X_pl, pipe.predict(X_pl), color=color, linestyle=style, linewidth=1, label=name)

plt.title(f'Polynomial Fit Comparison (Points={n_points}, Degree={degree}, Lambda={lam})')
plt.xlabel('x'); plt.ylabel('y'); plt.grid(True, alpha=0.3); plt.legend()
plt.show()
```

```
Unoptimized (OLS): y = 0.1102 - 1.1284*x^1 + 0.0283*x^2 + 0.5203*x^3 - 0.0703*x^4 - 0.0076*x^5 + 0.0106*x^6 | MSE: 0.0369
L1 (Lasso): y = 0.0434*x^5 - 0.0030*x^6 | MSE: 0.2373
L2 (Ridge): y = 0.1056 - 0.7304*x^1 + 0.0091*x^2 + 0.1823*x^3 - 0.0528*x^4 + 0.0537*x^5 + 0.0074*x^6 | MSE: 0.0486
```



P5 [20 points]

Choose a suitable way for formulating a composite objective to solve the following multiple multiobjective optimization

$$\text{minimize} \quad \boldsymbol{f}(\boldsymbol{x}) = (f_1(\boldsymbol{x}), ..., f_p(\boldsymbol{x}),$$

$$\text{subject to} \quad \sum_{i=1}^{n} x_i^2 \leq n^2,$$

where

$$f_i(\boldsymbol{x}) = x_i^2 + |x_{i+1}| \quad (i = 1, 2, ..., p-1), \qquad f_p(\boldsymbol{x}) = x_p^2 + |x_1|,$$

and

$$-n \leq x_i \leq n.$$

If you use the method of the weighted sum, does the optimal solution depend on the weighting coefficients?

Solution

Assumptions:

> minimize $\boldsymbol{f}(\boldsymbol{x}) = (f_1(\boldsymbol{x}), \ldots, f_p(\boldsymbol{x}))$
> subject to $\sum_{i=1}^{n} x_i^2 \leq n^2$
> $f_i(\boldsymbol{x}) = x_i^2 + |x_{i+1}| \quad (i = 1, 2, \ldots, p-1), \qquad f_p(\boldsymbol{x}) = x_p^2 + |x_1|$
> $-n \leq x_i \leq n$

$F(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_p f_p(x)$
$F(x) = \sum_{j=1}^{p} w_j f_j(x)$

The weighted sum method can be used to create a master function ($F(x)$) by multiplying each individual function (f_i) by a pre-determined weight. In this particular example, the optimal solution will not change with weights. Because it has squares (x^2) and absolute values ($|x|$) the minimum cost will be zero. Setting all variables to zero ($x = 0$) minimizes the equation regardless of weights.

```
# ANA670 - Week 3 HW - Q5
import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import minimize

w_3d = np.array([0.5, 0.3, 0.2])
print("=== 3D SURFACE PLOT DATA ===")
print(f"Fixed Weights Used: {w_3d}\nTheoretical Global Minimum: x = 0, F(x) = 0\n")

r = np.linspace(-1.5, 1.5, 60)
X1, X2 = np.meshgrid(r, r)
Z = np.zeros_like(X1)

for i in range(3): Z += w_3d[i] * (X1**2 + np.abs(X2) if i % 2 == 0 else X2**2 + np.abs(X1))

Z[X1**2 + X2**2 > 4] = np.nan
fig = plt.figure(figsize=(11, 4.5))
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
ax1.plot_surface(X1, X2, Z, cmap="viridis", alpha=0.8, edgecolor="none")
ax1.set_xlabel("$x_1$"); ax1.set_ylabel("$x_2$"); ax1.set_zlabel("$F(x)$"); ax1.set_title("n=2
Variable Surface")

n, p = 5, 5
obj = lambda x, w: sum(w[i] * (x[i % n] ** 2 + np.abs(x[(i + 1) % n])) for i in range(p))
vals = []

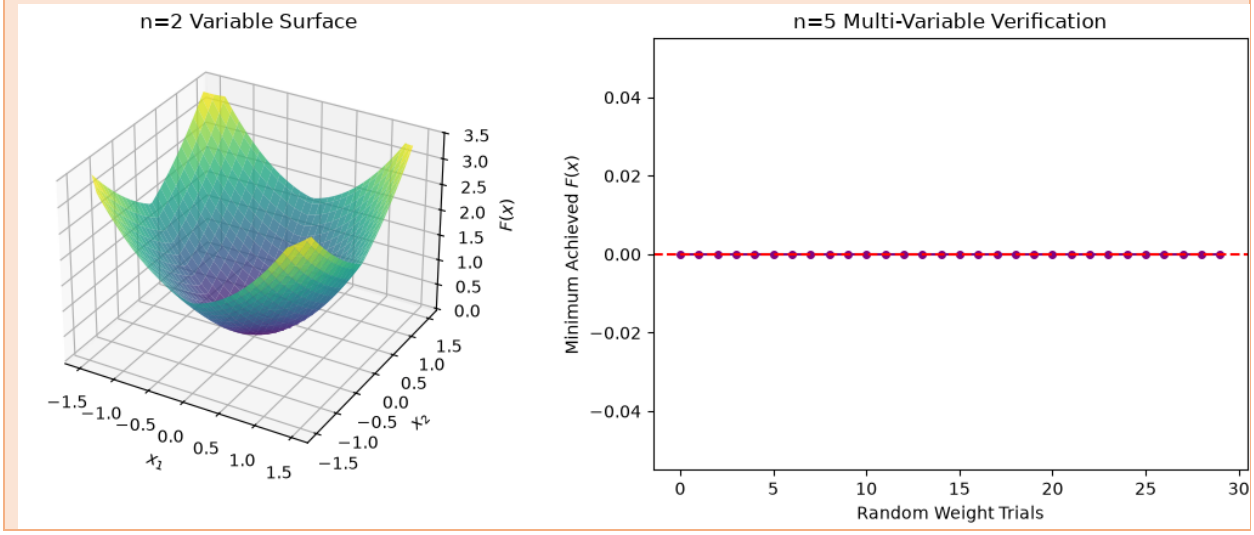
print("=== MULTI-VARIABLE VERIFICATION DATA (First 5 Trials) ===")
for trial in range(30):
    w = np.random.rand(p); w_norm = w / w.sum()
    res = minimize(obj, np.random.uniform(-1, 1, n), args=(w_norm,), method="SLSQP", bounds=[(-n,
n)] * n, constraints={"type": "ineq", "fun": lambda x: n**2 - np.sum(x**2)}, tol=1e-12)
    final_score = res.fun if res.fun > 1e-7 else 0.0
    vals.append(final_score)
```

```
    if trial < 5:
        print(f"Trial {trial+1}:")
        print(f"  Random Weights: {np.round(w_norm, 3)}\n  Resulting Knobs (x): {np.round(res.x, 5)}\n
Achieved Cost F(x): {res.fun:.7f}")

ax2 = fig.add_subplot(1, 2, 2)
ax2.plot(vals, "o-", color="purple", markersize=4); ax2.axhline(0, color="red", linestyle="--")
ax2.set_xlabel("Random Weight Trials"); ax2.set_ylabel("Minimum Achieved $F(x)$");
ax2.set_title("n=5 Multi-Variable Verification")
plt.tight_layout(); plt.show()
```

```
=== 3D SURFACE PLOT DATA ===
Fixed Weights Used: [0.5 0.3 0.2]
Theoretical Global Minimum: x = 0, F(x) = 0

=== MULTI-VARIABLE VERIFICATION DATA (First 5 Trials) ===
Trial 1:
  Random Weights: [0.398 0.291 0.14  0.099 0.073]
  Resulting Knobs (x): [-0. -0. -0. -0. -0.]
  Achieved Cost F(x): 0.0000000
Trial 2:
  Random Weights: [0.023 0.264 0.229 0.389 0.094]
  Resulting Knobs (x): [-0. -0. -0. -0. -0.]
  Achieved Cost F(x): 0.0000000
Trial 3:
  Random Weights: [0.062 0.22  0.188 0.272 0.257]
  Resulting Knobs (x): [-0. -0. -0. -0. -0.]
  Achieved Cost F(x): 0.0000000
Trial 4:
  Random Weights: [0.056 0.222 0.279 0.327 0.116]
  Resulting Knobs (x): [-0.  0. -0. -0. -0.]
  Achieved Cost F(x): 0.0000000
Trial 5:
  Random Weights: [0.439 0.235 0.109 0.021 0.196]
  Resulting Knobs (x): [-0. -0. -0.  0. -0.]
  Achieved Cost F(x): 0.0000000
```



P6 [20 points]

For multiobjective optimization, the understanding of the Pareto front is very important because the Pareto front can be easily converted to the optimal solutions when needed. Analyze the following optimization problem with two objectives

$$\text{minimize} \quad (f_1(x_1), f_2(\boldsymbol{x})),$$

where

$$\boldsymbol{x} = (x_1, ..., x_n), \qquad n = 30, \qquad x_i \in [0, 1],$$

and

$$f_1(x_1) = x_1, \quad f_2(\boldsymbol{x}) = g(x_2, ..., x_n)h(f_1(x_1), g(x_2, ..., x_n)),$$

with

$$g(x_2, ..., x_n) = 1 + 9\sum_{i=2}^{n} \frac{x_i}{(n-1)}, \quad h(f_1, g) = 1 - \sqrt{f_1/g}.$$

The Pareto optimal front is formed with g(x) = 1. Thus, sketch this front.

Solution

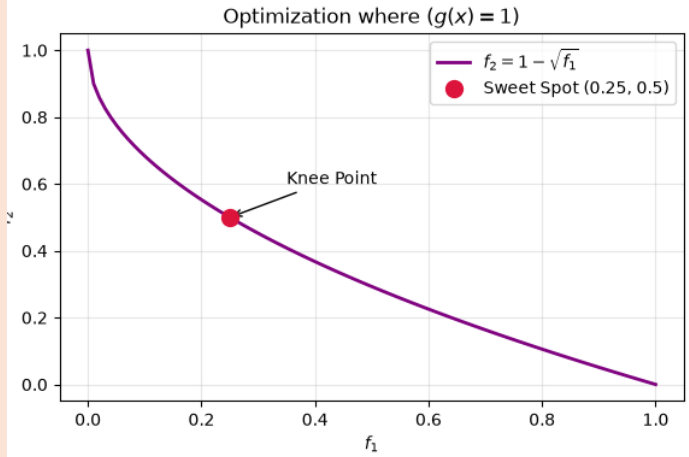
Assumptions:

> $f_1 = x_1$
> $f_2 = g \times h$
> $h = 1 - \sqrt{\frac{f_1}{g}}$
> $g = 1 + 9\sum_{i=2}^{n} \frac{x_i}{(n-1)} = 1$

$h = 1 - \sqrt{\frac{f_1}{g}} = 1 - \sqrt{\frac{f_1}{1}} = 1 - \sqrt{f_1}$
$f_2 = 1 - \sqrt{f_1}$

```
# ANA670 - Week 3 HW - Q6
import sympy as sp
f1_s = sp.Symbol("f1"); f2_s = 1 - sp.sqrt(f1_s)
kf1 = float(sp.solve(sp.Eq(sp.diff(f2_s, f1_s), -1), f1_s)[0]); kf2 = float(f2_s.subs(f1_s, kf1))
f1 = np.linspace(0, 1, 100); f2 = 1 - np.sqrt(f1)

plt.figure(figsize=(6, 4))
plt.plot(f1, f2, color="purple", lw=2, label=r"$f_2 = 1 - \sqrt{f_1}$")
plt.scatter(kf1, kf2, color="crimson", s=100, zorder=5, label=f"Sweet Spot ({kf1}, {kf2})")
plt.annotate("Knee Point", xy=(kf1, kf2), xytext=(kf1+0.1, kf2+0.1), arrowprops=dict(arrowstyle="->"))
plt.xlabel("$f_1$"); plt.ylabel("$f_2$"); plt.title("Optimization where ($g(x)=1$)")
plt.grid(True, alpha=0.3); plt.legend(); plt.tight_layout(); plt.show()
```


Optimization where $(g(x) = 1)$
$f_2 = 1 - \sqrt{f_1}$
Sweet Spot (0.25, 0.5)
Knee Point
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
f_1

The end